

Monitoring / observabilite

Stack d'observabilite de la plateforme MSPR HealthAI Coach (MSPR3 / TPRE601). Elle est fournie sous forme d'overlay Docker Compose (`docker-compose.monitoring.yml`) qui s'ajoute a la stack applicative sans la modifier.

Lancement

```
# Mode prod (images GHCR)
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d

# Mode dev (build local)
docker compose -f docker-compose.dev.yml -f docker-compose.monitoring.yml up -d
```

Arret de l'observabilite seule (sans toucher l'application) :

```
docker compose -f docker-compose.monitoring.yml down
```

Acces

Outil	URL	Role	Identifiants
Grafana	http://localhost:3001	Tableaux de bord (metriques + logs)	admin / admin (par default)
Prometheus	http://localhost:9090	Collecte de metriques + regles d'alerte	-
Alertmanager	http://localhost:9093	Alertes declenchees	-
Loki	http://localhost:3100	Logs centralises (interroges via Grafana)	-

Le mot de passe Grafana se surcharge via `GRAFANA_ADMIN_PASSWORD` dans `.env`.

Composants

Service	Image	Role
<code>prometheus</code>	<code>prom/prometheus</code>	Collecte et stocke les metriques (retention 7 jours), evalue les regles d'alerte
<code>alertmanager</code>	<code>prom/alertmanager</code>	Recoit et regroupe les alertes de Prometheus
<code>grafana</code>	<code>grafana/grafana</code>	Tableaux de bord, datasources Prometheus + Loki provisionnees
<code>loki</code>	<code>grafana/loki</code>	Stockage des logs (filesystem, retention 7 jours)
<code>promtail</code>	<code>grafana/promtail</code>	Collecte les logs de tous les conteneurs via le socket Docker
<code>cadvisor</code>	<code>cadvisor</code>	Metriques par conteneur (CPU, RAM, reseau, IO)
<code>node-exporter</code>	<code>prom/node-exporter</code>	Metriques de l'hote (CPU, RAM, disque, charge)
<code>postgres-exporter</code>	<code>postgres-exporter</code>	Metriques PostgreSQL metier (base <code>healthai</code>)
<code>postgres-exporter-auth</code>	<code>postgres-exporter</code>	Metriques PostgreSQL auth (base <code>auth_db</code>)
<code>mongodb-exporter</code>	<code>percona/mongodb_exporter</code>	Metriques MongoDB (base <code>reco_fitness</code>)

Donnees collectees (liste exhaustive)

Livable MSPR3 : documentation du systeme de supervision incluant la liste exhaustive des donnees collectees.

Metriques d'infrastructure (actives des le lancement)

Source	Exemples de donnees	Niveau
cAdvisor	<code>container_cpu_usage_seconds_total</code> , <code>container_memory_usage_bytes</code> , <code>container_network_receive_bytes_total</code> , <code>container_fs_usage_bytes</code>	par conteneur
node-exporter	<code>node_cpu_seconds_total</code> , <code>node_memory_MemAvailable_bytes</code> , <code>node_filesystem_avail_bytes</code> , <code>node_load1</code>	hote
postgres-exporter (x2)	<code>pg_up</code> , <code>pg_stat_database_*</code> (connexions, commits, rollbacks), <code>pg_database_size_bytes</code>	bases metier + auth
mongodb-exporter	<code>mongodb_up</code> , <code>mongodb_connections</code> , <code>mongodb_op_counters_total</code> , taille des collections	MongoDB
Prometheus	<code>up</code> (etat de chaque cible scrappee)	toutes cibles

Logs (actifs des le lancement)

Promtail collecte la sortie standard et d'erreur de **tous les conteneurs** et les pousse vers Loki, etiquetes par `container` , `stream` (stdout/stderr) et `service` . Interrogeables dans Grafana (Explore ou panneau "Logs recents").

Metriques applicatives (instrumentees, sprint A2)

Service	Endpoint	Donnees
API Spring Boot	<code>/api/actuator/prometheus</code>	<code>http_server_requests_seconds</code> (latence, debit, codes HTTP), JVM (heap, GC, threads), pool de connexions
AI-Nutrition (FastAPI)	<code>/metrics</code>	requetes HTTP par route, latence, erreurs (RED)
Reco-Fitness (FastAPI)	<code>/metrics</code>	requetes HTTP par route, latence, erreurs (RED)
AUTH (Hono/Bun)	<code>/metrics</code>	requetes HTTP, latence, compteurs (RED)

Les 4 cibles sont actives dans `prometheus/prometheus.yml` . L'endpoint Prometheus de l'API est ouvert sans JWT dans la config de securite (scraping interne) ; les endpoints `/metrics` des FastAPI et d'Auth sont publics sur le reseau Docker, acceptable en environnement de demonstration.

Important : ces metriques applicatives ne sont visibles qu'apres reconstruction des images des services (l'instrumentation vit dans le code source des repos). En mode dev : `docker compose -f docker-compose.dev.yml -f docker-compose.monitoring.yml up -d --build` . En mode prod : apres publication des nouvelles images GHCR par la CI.

Alertes basiques

Definies dans `prometheus/alerts.yml` :

Alerte	Condition	Severite
<code>CibleInjoignable</code>	<code>up == 0</code> pendant 1 min	critical
<code>ConteneurCpuEleve</code>	<code>> 0.9</code> coeur CPU pendant 5 min	warning
<code>ConteneurMemoireElevee</code>	<code>> 2 Go</code> de RAM pendant 5 min	warning

Tableau de bord

Le dashboard "MSPR HealthAI - Vue stack" est provisionne automatiquement dans Grafana (dossier "MSPR HealthAI") : cibles UP, disponibilite par job, CPU et memoire par conteneur, et flux de logs. Des dashboards par service seront ajoutes au sprint A3.